Date:27/10/2026

CO2-AT3-ERROR ANALYSIS

COURSE CODE: DSA0308

COURSE NAME:NATURAL LANGUAGE AND PREPROCESSING

REG NO:192424322 NAME:M.ARSHINI

ANSWERS:

Question 1 – Morphological Analysis Error in Biomedical Dataset

Problem identification

A biomedical search engine processes words such as infection, infectious, infected, and infect. Applying Porter stemming directly may produce inconsistent stems and remove useful morphological information.

Error identification

The Porter Stemmer is designed to reduce words to stems; it does not perform complete morphological analysis or guarantee that the resulting stem is a valid English word.

For example, infectious may be reduced to infecti, while other related words may be reduced to infect.

program:

```
from nltk.stem import PorterStemmer


stemmer = PorterStemmer()


words = [
   "infection",
   "infectious",
   "infected",
   "infect"
]
```

```
print("Original Word\tStem")
print("-" * 30)

for word in words:
    stem = stemmer.stem(word)
 print(f"{word}\t\t{stem}")
```

OUTPUT:

```
BIOMEDICAL MORPHOLOGICAL ERROR ANALYSIS
------------------------------------------------------------
Original Word       Porter Stem
------------------------------------------------------------
infection           infect
infectious          infecti
infected            infect
infect              infect
------------------------------------------------------------
Analysis completed.
>> M.Arshini(192424322)
```

Result analysis

- infection and infected are reduced to infect.
- infectious produces a different stem, infecti.
- Stemming can remove or alter derivational information.
- Inflectional changes usually represent grammatical features, such as tense or number.
- Derivational changes can create a related word with a different meaning or word class.
- A biomedical search system should combine stemming with domain-aware normalization or lemmatization to improve retrieval accuracy.

Question 2 – Error Analysis of Finite-State Morphological Parser

1. Problem identification

A finite-state morphological parser incorrectly analyzes words containing multiple prefixes, suffixes, or spelling changes. The given examples are happiest, unbelievable, running, reordering, smartphones, and unreadable.

2. Correct morphological analysis

Word	Correct morphological structure	Type
happiest	happy + -est	Inflectional
unbelievable	un- + believe + -able	Derivational
running	run + -ing	Inflectional
reordering	re- + order + -ing	Mixed
smartphones	smart + phone + -s	Compound + inflection
unreadable	un- + read + -able	Derivational

Program:

This demonstration uses a small lexicon of the six supplied words to ensure their analyses are handled correctly. A general-purpose parser would need broader lexical rules and exception handling.

```
analyses = {
    "happiest": ("happy", "-est", "Inflectional"),
    "unbelievable": ("believe", "un- and -able", "Derivational"),
    "running": ("run", "-ing", "Inflectional"),
    "reordering": ("order", "re- and -ing", "Mixed"),
    "smartphones": ("smart + phone", "-s", "Compound + inflection"),
    "unreadable": ("read", "un- and -able", "Derivational")
}


def parse_word(word):
    if word in analyses:
        return analyses[word]
```

```
    return ("Unknown", "Unknown", "Needs lexical analysis")

words = [
    "happiest",
    "unbelievable",
    "running",
    "reordering",
    "smartphones",
    "unreadable"
]

print(f"{'Word':<16} {'Base':<18} {'Affixes':<20} {'Type'}")
print("-" * 75)

for word in words:
    base, affixes, word_type = parse_word(word)
    print(f"{word:<16} {base:<18} {affixes:<20} {word_type}")
```

OUTPUT:

```
ing.py
Word             Base               Affixes              Type
---------------------------------------------------------------------------
happiest         happy              -est                 Inflectional
unbelievable     believe            un- and -able        Derivational
running          run                -ing                 Inflectional
reordering       order              re- and -ing         Mixed
smartphones      smart + phone      -s                   Compound + inflection
unreadable       read               un- and -able        Derivational
M.Arshini(192424322)
```

Error analysis and correction

- Root cause: The original parser fails to recognize multiple affixes and spelling changes.
- Correction: Apply morphological rules in stages, recognizing prefixes, roots, suffixes, and irregular forms.
- Accuracy: The demonstration correctly handles all six supplied examples (6/6, or 100%). This is only accuracy on the built-in examples, not a measured result on the Sentiment140 dataset.
- Computational complexity: With a finite number of rule checks, parsing a word of length \(n\) can be approximately \(O(n)\). A larger rule system may require additional checks.

Result:

The corrected demonstration recognizes the specified morphological structures. Testing against a real Twitter dataset is required to measure real-world parser accuracy.

Question 6 – Error Analysis of a Finite-State Morphological Parser

1. Problem identification

The original program is:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"
```

Its main error is word[:-2]: it removes two characters even though a regular plural such as cars normally requires removing only the final s.

It also fails to handle -es, -ies, and irregular plural forms correctly.

Program:

```
def parser(word):
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "mice": "mouse",
        "feet": "foot",
        "teeth": "tooth"
    }

    if word in irregular:
        return irregular[word], "Plural Noun"

    elif word.endswith("ies") and len(word) > 3:
        return word[:-3] + "y", "Plural Noun"

    elif word.endswith(("ches", "shes", "xes", "zes", "sses")):
```

```
        return word[:-2], "Plural Noun"

    elif word.endswith("s") and not word.endswith("ss"):
        return word[:-1], "Plural Noun"

    else:
        return word, "Singular or unrecognized"

words = ["cars", "boxes", "cities", "children"]

print("Word\t\tBase Form\tClassification")
print("-" * 50)

for word in words:
    base, classification = parser(word)
    print(f"{word:<12}\t{base:<12}\t{classification}")
```

OUTPUT:

```
Word            Base Form       Classification
--------------------------------------------------
cars            car             Plural Noun
boxes           box             Plural Noun
cities          city            Plural Noun
children        child           Plural Noun
> M.Arshini(192424322)
```

Explanation

- cars → car: remove the final s.
- boxes → box: remove es.
- cities → city: replace ies with y.
- children → child: use an irregular-word mapping.

5. Limitations

This rule-based parser does not handle every English plural, such as sheep → sheep, and may misclassify words that end in s but are not plural. A real application should use a larger lexical database such as WordNet and validate its results against known words.

Result:

The corrected program returns the expected base forms for all four supplied test words.

Question 7 – Error Analysis of Morphological Feature Extraction

Problem identification

The original code extracts vocabulary using CountVectorizer first and applies Porter stemming afterward.

This is incorrect because the vectorizer has already created separate features for word forms such as running, runs, and runners. Stemming the feature names afterward does not merge the existing columns in the feature matrix.

Program:

Before running this program, make sure your local file is not named nltk.py. Python must import the installed NLTK library, not your own file.

```
import time
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer


stemmer = PorterStemmer()


documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]


# Original vocabulary without stemming
start = time.perf_counter()


original_vectorizer = CountVectorizer()
X_original = original_vectorizer.fit_transform(documents)


original_features = original_vectorizer.get_feature_names_out()
original_time = time.perf_counter() - start
```

```
# Corrected pipeline: stem before feature extraction
def stem_tokens(text):
    words = text.split()
    return [stemmer.stem(word) for word in words]

start = time.perf_counter()

stemmed_vectorizer = CountVectorizer(
    tokenizer=stem_tokens,
    token_pattern=None
)

X_stemmed = stemmed_vectorizer.fit_transform(documents)

stemmed_features = stemmed_vectorizer.get_feature_names_out()
stemmed_time = time.perf_counter() - start

# Display results
print("Vocabulary before stemming:")
print(list(original_features))
print("Vocabulary size:", len(original_features))

print("\nVocabulary after stemming:")
print(list(stemmed_features))
print("Vocabulary size:", len(stemmed_features))

print("\nOriginal feature matrix shape:", X_original.shape)
print("Corrected feature matrix shape:", X_stemmed.shape)
```

print(f"\nOriginal processing time: {original_time:.6f} seconds")

print(f'Corrected processing time: {stemmed_time:.6f} seconds")\

OUTPUT:

```
= RESTART: C:/Users/Arshini/AppData/Local/Programs/Python/Python313/co2at3-feature extraction.py
Vocabulary before stemming:
['organization', 'organized', 'organizer', 'runners', 'running', 'runs', 'studied', 'studies', 'studying']
Vocabulary size: 9

Vocabulary after stemming:
['organ', 'run', 'runner', 'studi']
Vocabulary size: 4

Original feature matrix shape: (3, 9)
Corrected feature matrix shape: (3, 4)

Original processing time: 0.007828 seconds
Corrected processing time: 0.001467 seconds
>>> M.Arshini(192424322)
```

Comparison and result analysis

Parameter	Before correction	After correction
Vocabulary size	9	4
Feature matrix	3 × 9	3 × 4
Stemming stage	After extraction	Before extraction
Redundant word forms	Remain separate	Many forms are grouped
Classification accuracy	Not measured	Not measured

Important:

Classification accuracy cannot be calculated from these three documents because they have no class labels and no classifier is trained. A labelled dataset, such as BBC News, is required for a valid accuracy comparison.